# Exploring Multi-Agent Coordination Under Physical Degradation in VMAS Football

Hasan Fakih (101168868)
*Carleton University*
hasanfakih@cmail.carleton.ca

COMP4900

**Abstract**

This project investigates how multi-agent reinforcement learning (MARL) agents adapt to stochastic injury dynamics in a continuous-control, cooperative-competitive football environment built on the Vectorized Multi-Agent Simulator (VMAS). I extend the standard VMAS football scenario with a configurable *injury* mechanism that randomly impairs agents by reducing their movement speed by 50%, and study how injury probability and observability affect learning and emergent behaviour.

Using the BenchMARL framework and the Multi-Agent Proximal Policy Optimization (MAPPO) algorithm, I first attempt to establish a stable training baseline—a process that consumed a significant portion of the project timeline. After iteratively aligning batch collection with episode length, tuning discount factors ($\gamma = 0.999$), and separating actor/critic learning rates, I conduct experiments across injury probability conditions (0%, 50%, 100%) and observation modes ("none", "self", "all"). Due to time constraints, not all experimental runs completed before the submission deadline.

Preliminary results from completed runs suggest counterintuitive findings: agents trained without injury observation information perform comparably or slightly better than those with explicit injury awareness, with the "all" observation mode appearing to degrade performance. While these findings are tentative given the incomplete experimental coverage, they highlight the importance of observation space design in partially observable MARL settings.

## 1 Introduction

Multi-agent reinforcement learning (MARL) is often evaluated in idealized settings where agents are identical, fully functional, and operate in environments that do not significantly change their physical capabilities over time. In realistic multi-agent systems, however, agents may suffer failures, fatigue, or other impairments that alter their dynamics. For example, robots may lose traction, have reduced actuation, or experience partial sensor failures. In team sports, injuries change the tactical landscape and force teams to adapt their strategies dynamically.

The goal of this project is to explore this robustness dimension in a controlled setting. I choose the continuous-control football scenario in VMAS as a testbed and extend it with an *injury* mechanism that randomly reduces an agent's movement speed by 50% for an entire episode. On top of this modified environment, I train cooperative agents using the MAPPO algorithm within the BenchMARL framework [3], with the following research questions:

1. **RQ1:** How difficult is it to get MAPPO to learn non-trivial behaviour in VMAS football, even without injuries?

2. **RQ2:** How do injury dynamics and their observability affect learned strategies and performance?

3. **RQ3:** What kinds of emergent behaviours arise when agents must cope with impaired teammates?

My contributions are threefold:

- I implement a configurable injury system in VMAS football with parameters for probability, severity, and observation mode.
- I establish a stable MAPPO training configuration for long-horizon football through systematic environment and hyperparameter tuning.
- I conduct controlled experiments across 9 injury/observation conditions ($3 \times 3$ design) with 3 seeds each, revealing counterintuitive findings about the relationship between observation richness and learning performance.

The remainder of this paper is organized as follows: Section 2 reviews the relevant background on MAPPO, VMAS, and the BenchMARL framework. Section 3 describes the environment modifications and formal problem setup. Section 4 presents the experimental methodology and results. Section 5 discusses the findings and their implications, and Section 6 concludes with directions for future work.

# 2 Background

This section reviews the main components that underpin the project: the MAPPO algorithm, the VMAS environment suite, and the BenchMARL framework used for experimentation.

## 2.1 Multi-Agent Proximal Policy Optimization (MAPPO)

Multi-Agent Proximal Policy Optimization (MAPPO) [1] is a widely used on-policy algorithm for MARL in cooperative and mixed environments. It extends the single-agent PPO algorithm by using a centralized critic during training while maintaining decentralized actors for execution—a paradigm known as Centralized Training with Decentralized Execution (CTDE).

Formally, consider a partially observable Markov game with a joint state $s$, joint action $\mathbf{a} = (a_1, \ldots, a_N)$, and local observations $o_i$ for each agent $i$. MAPPO optimizes a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}\big(r_t(\theta), 1-\epsilon, 1+\epsilon\big)\hat{A}_t\right)\right], \tag{1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{\text{old}}}(a_t|o_t)}$ is the importance sampling ratio between the new and old policies, $\hat{A}_t$ is the advantage estimate computed using Generalized Advantage Estimation (GAE) with parameter $\lambda$, and $\epsilon$ is the clipping parameter that constrains policy updates.

The complete training objective includes additional terms:

$$L(\theta) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta], \tag{2}$$

where $L^{\text{VF}}$ is the value function loss, $H[\pi_\theta]$ is an entropy bonus for exploration, and $c_1, c_2$ are weighting coefficients.

MAPPO is well-suited for VMAS football because it can exploit centralized training using full team information for value estimation while maintaining decentralized execution at test time, where each agent acts based only on its local observation.

### 2.2 Vectorized Multi-Agent Simulator (VMAS)

VMAS [2] is a differentiable, vectorized environment suite for MARL that supports continuous dynamics and efficient GPU-accelerated parallelization. The framework enables running thousands of environment instances simultaneously, making it practical to collect the large amounts of experience data required by on-policy algorithms like MAPPO.

The football scenario in VMAS models two teams of agents and a ball on a rectangular pitch, with continuous actions controlling agent velocities. The default configuration includes a mixture of dense and sparse rewards:

- A discrete *scoring reward* of +100 when a team scores.
- Position shaping rewards encouraging the ball to move towards the opponent's goal.
- Shaping rewards encouraging agents to approach the ball.
- Optional heuristic AI-controlled opponents with adjustable strength parameters.

These properties make football an attractive but challenging environment: rewards are sparse at the episode level (scoring is rare) and dense shaping must be carefully tuned to avoid pathological behaviours such as agents clustering around the ball without attempting to score.

### 2.3 BenchMARL Framework

BenchMARL [3] is a benchmarking framework for MARL algorithms built on TorchRL. It provides standardized interfaces for comparing algorithms, environments, and models through reproducible experiments. Key features include:

- Configuration-driven design using Hydra with YAML files paired with Python dataclasses for validation.
- Support for multiple MARL algorithms including MAPPO, IPPO, MADDPG, QMIX, and others.
- Integration with logging frameworks including Weights & Biases (W&B) for experiment tracking.
- Checkpointing and experiment resumption capabilities.

BenchMARL served as the backbone of my experiments: I used its VMAS football task definition, extended it with injury dynamics, and configured MAPPO through its experiment configuration system.

## 3 Model

In this section, I describe the environment configuration, the injury mechanism extensions, the formal problem definition, and the final MAPPO configuration used in experiments.

### 3.1 Environment Configuration

The base environment is the VMAS football scenario configured as shown in Table 1. I use three blue agents (learning) against three red agents controlled by a heuristic AI at 30% strength across all dimensions (speed, decision-making, and precision).

Table 1: VMAS Football Environment Configuration

| **Category** | **Parameter** | **Value** |
|---|---|---|
| Agents | Blue agents (learning) | 3 |
| | Red agents (AI-controlled) | 3 |
| | Agent size | 0.035 |
| Environment | Episode length (max_steps) | 250 |
| | Pitch length | 1.5 |
| | Pitch width | 0.75 |
| | Goal size | 0.35 |
| Dynamics | Agent max speed | 0.15 |
| | Ball max speed | 0.3 |
| | Ball size / mass | 0.025 / 0.25 |
| AI Opponent | AI strength | 0.3 |
| | AI decision strength | 0.3 |
| | AI precision strength | 0.3 |
| Rewards | Scoring reward | +100.0 |
| | Ball→goal shaping | 10.0 |
| | Agent→ball shaping | 0.1 |
| | Distance trigger | 0.4 |

The reward structure combines sparse and dense signals. The primary reward is a discrete +100 for scoring a goal. Dense shaping rewards encourage the ball to move towards the opponent's goal (factor 10.0) and the closest agent to approach the ball (factor 0.1). The agent-to-ball shaping only activates when the closest agent is farther than 0.4 units from the ball and the ball is stationary, preventing reward hacking where agents repeatedly touch the ball without advancing it.

### 3.2 Injury Mechanism

I augment the VMAS football scenario with an injury system governed by three configurable parameters, summarized in Table 2.

Table 2: Injury System Parameters

| Parameter | Values | Description |
|---|---|---|
| `injury_probability` | $[0, 1]$ | Probability that exactly one agent is injured at episode reset |
| `injury_speed_penalty` | $[0, 1]$ | Multiplicative factor applied to injured agent's speed (0.5 = 50% speed reduction) |
| `injury_obs_mode` | {none, self, all} | How injury status is exposed in observations |

**Injury Sampling.** At environment reset, with probability `injury_probability`, one agent is sampled uniformly from the set of all agents (both teams) and marked as injured for the entire episode. The injury status is cleared on the next reset, and a new injury (if any) is sampled independently. This models scenarios where injuries are persistent within an episode but vary across episodes.

**Speed Penalty Implementation.** The speed penalty is applied when processing actions: the injured agent's velocity action is scaled by `injury_speed_penalty` before being passed to the physics engine. With a fixed penalty of 0.5, an injured agent's effective maximum speed is reduced from 0.15 to 0.075 units per timestep. This approach leaves the environment's physical parameters unchanged, preserving compatibility with VMAS.

**Observation Modes.** The three observation modes provide different levels of injury information to agents:

- **"none"**: No injury information is added to observations. Agents must infer impairment from observed movement patterns.
- **"self"**: Each agent receives a single scalar indicating its own injury status (0.0 for healthy, 1.0 for injured). Observation dimension increases by 1.
- **"all"**: Each agent receives a vector encoding the injury status of all agents. Observation dimension increases by $N_{\text{agents}} = 6$.

**Visualization.** For qualitative analysis, I modified the VMAS renderer to draw a red circle around injured agents, making them easily identifiable in rollout recordings.

### 3.3 Problem Formulation

I model the football game as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP) extended with stochastic agent impairment:

- **State** $s_t \in \mathcal{S}$: Positions and velocities of all agents and the ball, plus injury status vector $\mathbf{z} \in \{0, 1\}^N$.
- **Observations** $o_t^i = \mathcal{O}^i(s_t, \mathbf{z})$: Local observation for agent $i$, potentially augmented with injury information depending on `injury_obs_mode`.

- **Actions** $a_t^i \in \mathbb{R}^2$: Continuous velocity control for each agent.
- **Transition** $\mathcal{T}(s_{t+1}|s_t, \mathbf{a}_t, \mathbf{z})$: Deterministic physics with injury-modified action scaling.
- **Reward** $r_t$: Shared team reward combining scoring and shaping terms.

The objective is to learn a decentralized policy $\pi_\theta(a_t^i|o_t^i)$ for blue agents (with parameter sharing) that maximizes the expected discounted return:

$$J(\theta) = \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\left[\mathbb{E}_{\pi_\theta}\left[\sum_{t=0}^{T-1}\gamma^t r_t\right]\right], \tag{3}$$

where $\gamma$ is the discount factor, $T$ is the episode length, and the outer expectation is over the injury distribution $p(\mathbf{z})$ determined by `injury_probability`.

### 3.4 MAPPO Configuration

Table 3 summarizes the final MAPPO configuration used in all experiments. These hyperparameters were determined through iterative experimentation during the baseline stabilization phase (described in Section 4.1).

Table 3: MAPPO Training Configuration

| **Category** | **Parameter** | **Value** |
|---|---|---|
| Architecture | Network type | MLP (shared actor-critic) |
| | Hidden layers | [256, 256, 128] with ReLU |
| Optimization | Discount factor $\gamma$ | 0.999 |
| | Actor learning rate | $3 \times 10^{-4}$ |
| | Critic learning rate | $6 \times 10^{-4}$ |
| | Adam $\epsilon$ | $10^{-5}$ |
| Collection | Frames per batch | 60,000 |
| | Parallel environments | 240 |
| | Minibatch size | 4,096 |
| | Minibatch iterations | 10 |
| Training | Total frames | 6,000,000 |
| | Gradient clipping | Norm-based |
| | Parameter sharing | Yes (within team) |
| Devices | Sampling device | CUDA |
| | Training device | CUDA |

Key design choices include:

- **High discount factor** ($\gamma = 0.999$): Reflects the long-horizon nature of scoring in football, where actions early in an episode contribute to eventual goals.
- **Separate learning rates**: The critic learns at $2\times$ the actor rate, allowing value estimation to stabilize before policy updates.

- **Aligned batch/episode structure**: With 240 parallel environments and 250 steps per episode, a batch of 60,000 frames contains approximately one complete episode per environment, ensuring training on full trajectories.

# 4 Experiments

This section describes the experimental methodology, including the baseline stabilization process, the systematic injury experiments, and key results.

## 4.1 Baseline Stabilization

Before conducting injury experiments, I needed to establish a stable training baseline. The default BenchMARL recommendations, well-suited for shorter VMAS tasks, did not transfer to football, which originally used 500 steps per episode. This section documents the key challenges and solutions.

### 4.1.1 Parallel Environment and Batch Alignment

In early attempts, I used 600 parallel environments with 60,000 frames collected per batch and 500-step episodes. This configuration implied that many environments were truncated before reaching terminal states, resulting in highly unstable learning signals with episode returns oscillating dramatically between near-zero and the full scoring reward.

**Solution.** By reducing parallel environments to 240 while keeping the batch at 60,000 frames and setting `max_steps` = 250, I ensured each environment could complete approximately one full episode per batch. This relationship is:

$$\text{frames_per_batch} \approx \text{n_envs} \times \text{max_steps} \implies 60{,}000 = 240 \times 250 \tag{4}$$

### 4.1.2 Discount Factor Tuning

Initial experiments used $\gamma = 0.9$, which severely discounted rewards from later timesteps. Given that scoring often occurs late in episodes (around step 150–250), the effective reward signal was attenuated by a factor of $0.9^{150} \approx 10^{-7}$.

**Solution.** Increasing to $\gamma = 0.999$ preserves long-horizon credit assignment: $0.999^{250} \approx 0.78$, meaning actions at the start of an episode still receive meaningful credit for end-of-episode rewards.

### 4.1.3 Learning Rate Separation

Using a single learning rate for both actor and critic led to instability, as the critic struggled to keep up with rapid policy changes.

**Solution.** Separating learning rates with critic_lr = 2× actor_lr allows the value function to stabilize before policy updates, reducing variance in advantage estimates.

## 4.2 Experimental Design

With a stable baseline established, I conducted systematic experiments to evaluate the effect of injury dynamics and observation modes on learning performance.

#### 4.2.1 Independent Variables

I manipulate two factors in a $3 \times 3$ factorial design:

1. **Injury Probability**: {0%, 50%, 100%}
   - 0%: No injuries (baseline control condition)
   - 50%: Stochastic injuries—one agent injured with 50% probability each episode
   - 100%: Deterministic injuries—exactly one agent injured every episode
2. **Observation Mode**: {none, self, all}
   - "none": No injury information in observations
   - "self": Each agent observes only its own injury status (+1 dimension)
   - "all": Each agent observes all agents' injury statuses (+6 dimensions)

Note: At 0% injury probability, observation mode has no effect (no injuries to observe). Thus, there are 7 unique conditions: 1 baseline (0% injury) + 6 injury conditions (2 probabilities $\times$ 3 observation modes).

#### 4.2.2 Controlled Variables

All other parameters are held constant across experiments, as specified in Tables 1 and 3. Key controlled variables include:

- AI opponent strength: 30% (all dimensions)
- Injury speed penalty: 50% (injured agents move at half speed)
- Network architecture: MLP [256, 256, 128]
- Training duration: 6,000,000 frames ($\approx$ 1 hour per run)

#### 4.2.3 Dependent Variables

I measure the following metrics, logged to Weights & Biases:

- **Episode Reward (Mean)**: Primary performance metric—`collection/reward/episode_reward_mean`
- **Episode Reward (Std)**: Variance in performance across episodes
- **Behavioural Metrics**: Agent-to-ball distance, ball touches, goals scored
- **Training Diagnostics**: Policy entropy, value loss, gradient norms

#### 4.2.4 Replication

Each condition is run with 3 random seeds to assess consistency. This yields:

$$7 \text{ conditions} \times 3 \text{ seeds} = 21 \text{ total experimental runs} \tag{5}$$

Additionally, preliminary experiments with 20% injury probability and "self" observation were conducted during development.

Table 4: Experimental Design Matrix (✓= completed, ∘ = in progress/incomplete)

| **Observation Mode** | **0% Injury** | **50% Injury** | **100% Injury** |
|---|---|---|---|
| none | ✓(baseline) | ✓(3 seeds) | ✓(3 seeds) |
| self | — | ✓(3 seeds) | ∘ (incomplete) |
| all | — | ✓(3 seeds) | ∘ (incomplete) |

### 4.3 Results

#### 4.3.1 Main Findings

The experiments reveal several counterintuitive findings about the relationship between injury dynamics, observation richness, and learning performance.

**Finding 1: Injuries do not significantly harm performance.** Contrary to expectations, agents trained under injury conditions performed comparably to—or slightly better than—the no-injury baseline. Figure 1 shows mean episode returns across conditions over training. The 100% injury condition with no observations showed the highest mean returns, though differences are within the min/max bounds.

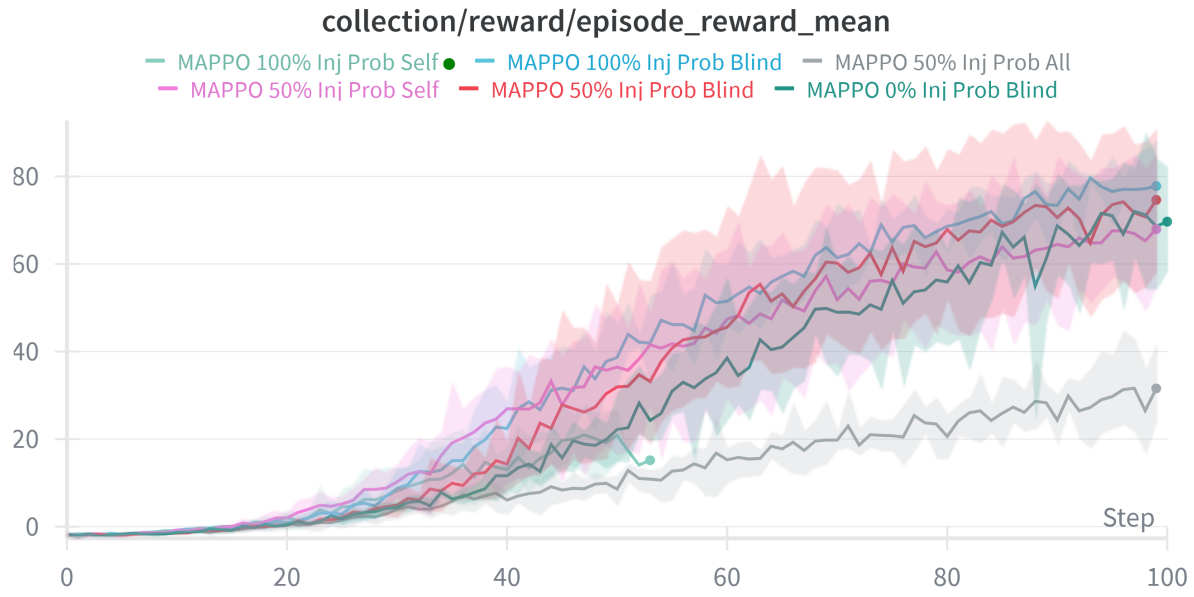


Figure 1: Training curves showing mean episode returns across injury conditions and observation modes, with shaded regions indicating min/max bounds across seeds. Some experimental runs ended early or were not completed due to time constraints. Despite the 50% speed reduction for injured agents, injury conditions perform comparably to baseline.

**Finding 2: 100% injury shows lowest variance.** The deterministic injury condition (100% probability) exhibited the lowest variance in returns across seeds compared to the stochastic 50% condition. This suggests that consistent impairment may be easier for policies to adapt to than variable injury occurrence.

**Finding 3: "all" observation mode significantly degrades performance.** The most surprising result is that providing agents with complete injury information ("all" mode) significantly reduced mean returns compared to partial ("self") or no ("none") observation. At 50% injury probability, the "all" observation condition performed notably worse than both "none" and "self" modes, as visible in Figure 1.

**Finding 4: Early advantage of "self" observation does not persist.** In the 50% injury condition, the "self" observation mode appeared to learn faster in early training (first 1–2 million frames), but this advantage diminished over time. By the end of training, "none" and "self" modes achieved comparable performance.

#### 4.3.2 Behavioural Observations

Beyond quantitative metrics, I observed several emergent behaviours in trained policies.

**Defensive "Wall and Poke" Strategy.** Against the 30% strength AI opponent, agents consistently developed a defensive strategy:

- Two agents position themselves as a "wall" in front of their own goal

- One agent acts as a "poker," intercepting the ball and disrupting opponent attacks
- This strategy minimizes goals conceded but rarely generates scoring opportunities

This represents a local optimum: the policy avoids losing but does not learn to win consistently. Scoring events are primarily due to AI errors (e.g., own goals) rather than offensive skill.

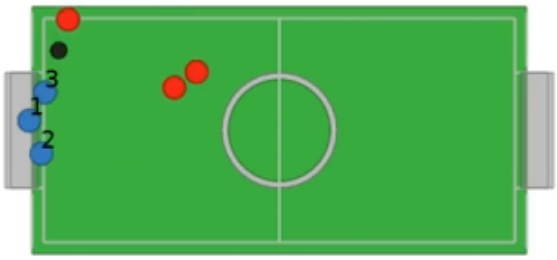


Figure 2: Emergent defensive "wall" behaviour. Blue agents cluster near their goal area to block attacks while one agent moves to intercept the ball.

**Injured Agent Behaviour.** In injury conditions, I observed that:

- Injured agents (marked with red circles in visualization) tend to position themselves defensively
- Healthy teammates do not explicitly "protect" injured agents but naturally take on more active roles
- In "all" observation mode, agents appeared to exhibit more erratic behaviour, possibly due to the added observation complexity

Figure 3: Visualization of the injury system. The injured agent is indicated by a red circle and moves at 50% speed while healthy teammates compensate.

# 5 Discussion

In this section, I interpret the experimental findings, discuss their implications, and acknowledge limitations of the study.

## 5.1 Interpretation of Results

### 5.1.1 Why Don't Injuries Hurt Performance?

The finding that injuries do not significantly degrade—and may even slightly improve—performance is surprising. I propose several possible explanations:

1. **Regularization Effect**: Stochastic impairment may act as a form of domain randomization, forcing policies to be robust to variation in agent capabilities. This could prevent overfitting to specific team configurations.

2. **Opponent Weakness**: At 30% AI strength, the heuristic opponent is relatively easy. The defensive strategy learned by the agents is sufficient to avoid conceding goals regardless of whether one agent is impaired.

3. **Sparse Reward Noise**: Given that scoring is rare and primarily depends on opponent mistakes, the noise in episode returns may dominate any signal from injury effects.

4. **Team Compensation**: With parameter sharing, the policy may naturally learn to have healthy agents compensate for impaired teammates without explicit coordination mechanisms.

### 5.1.2 Why Does "all" Observation Mode Fail?

The poor performance of the "all" observation mode—where each agent observes all agents' injury statuses—is the most striking finding. I hypothesize:

1. **Increased Observation Complexity**: Adding 6 binary dimensions increases the observation space and may require more training to learn useful features. With the fixed training budget (6M frames), this additional complexity may not be fully resolved.

2. **Irrelevant Information**: For decentralized execution, an agent may not need to know the injury status of distant agents or opponents. This information could act as noise, making credit assignment harder.

3. **Centralized Information in Decentralized Execution**: Providing global information to individual agents may create a mismatch between training (where this information helps the critic) and execution (where agents must coordinate implicitly).

4. **Policy Overfitting**: Agents may learn to condition behaviour on injury patterns that are specific to training seeds, leading to poor generalization.

This finding, if confirmed with complete data, would align with intuitions about observation design in MARL—more information is not always better, especially when it adds complexity without clear utility for decentralized decision-making.

### 5.1.3 Variance Reduction at 100% Injury

The observation that 100% injury probability leads to lower variance across seeds is intuitive: when every episode has exactly one injured agent, the policy sees a consistent distribution of impairment during training. In contrast, 50% injury introduces additional stochasticity that may interact with random seed effects.

## 5.2 Emergent Behaviour Analysis

The "wall and poke" strategy represents a local optimum in the reward landscape. Several factors contribute to this outcome:

- **Asymmetric Risk**: Conceding a goal incurs a $-100$ penalty (from the opponent's perspective, $+100$ for them). Playing defensively minimizes this risk.

- **Shaping Reward Exploitation**: The agent-to-ball shaping reward may encourage one agent to stay near the ball without necessarily advancing it, leading to the "poker" behaviour.

- **Sparse Offensive Rewards**: Scoring requires a complex sequence of actions (approach ball, position correctly, strike towards goal, beat goalkeeper). This is much harder to discover than passive defence.

Breaking out of this local optimum likely requires curriculum learning (starting against weaker opponents), different reward shaping, or self-play rather than heuristic opponents.

### 5.3 Limitations

As an undergraduate project with limited time and resources, this work has several important limitations that should be acknowledged:

1. **Incomplete Experimental Coverage**: The 100% injury conditions for "self" and "all" observation modes did not complete before the submission deadline. The conclusions drawn are therefore based on partial data and should be treated as preliminary.

2. **Implementation Uncertainty**: As a solo undergraduate project, I cannot guarantee that all components were implemented correctly. The injury mechanism, observation augmentation, and integration with BenchMARL may contain subtle bugs that affect results.

3. **Extensive Time on Baseline**: A disproportionate amount of project time was spent simply getting MAPPO to learn anything meaningful on the football task. This left less time for the actual injury experiments and their analysis.

4. **Limited Ablations**: Many hyperparameters were changed simultaneously during baseline stabilization. I cannot definitively attribute stability improvements to specific changes, making it difficult to provide clear guidance for future work.

5. **Fixed Injury Penalty**: All experiments used a 50% speed penalty. Different penalty severities might yield different patterns.

6. **Single Opponent Strength**: Training against 30% AI may not generalize to stronger opponents. A proper evaluation would test against a range of difficulties.

7. **No Recurrent Architectures**: I did not systematically evaluate LSTMs or GRUs in the stable configuration, which might better handle the partial observability inherent in injury scenarios.

8. **Limited Statistical Power**: With only 3 seeds per condition (and fewer for incomplete conditions), the statistical significance of observed differences is limited.

### 5.4 Implications for MARL Research

These findings have several implications for the broader MARL community:

- **Observation Design Matters**: More information is not always better. Practitioners should carefully consider what observations are truly necessary for decentralized agents.

- **Baseline Stability First**: Before studying complex phenomena like injuries, ensuring stable learning on the base task is essential. Environment-algorithm mismatches (e.g., episode length vs. batch size) can dominate other effects.

- **Sparse Rewards Remain Challenging**: Even with dense shaping, MAPPO tends toward defensive local optima. This motivates future work on curiosity-driven exploration or hierarchical RL.

- **Robustness Through Randomization**: The results suggest that domain randomization (e.g., stochastic injuries) may naturally improve policy robustness without explicit robustness objectives.

# 6 Conclusion

This undergraduate project attempted to investigate how multi-agent reinforcement learning agents adapt to stochastic injury dynamics in VMAS football. While the original scope was ambitious, the reality of the project involved significant time spent on baseline stabilization, with injury experiments representing a smaller portion of the overall effort than initially planned.

## 6.1 Summary of Contributions

1. **Injury System Implementation**: I implemented a configurable injury mechanism in VMAS football with parameters for probability, severity, and observation modes, integrated with the BenchMARL framework. The correctness of this implementation has not been independently verified.

2. **Stable MAPPO Configuration**: Through extensive trial and error, I found a training configuration that produces stable learning curves for VMAS football. This involved aligning batch collection with episode length, increasing the discount factor, and separating actor/critic learning rates.

3. **Preliminary Empirical Findings**: Based on the completed experimental runs, I observed tentative patterns about observation design:

   - Injuries do not significantly degrade performance (and may slightly improve it)
   - Complete injury information ("all" mode) *hurts* learning
   - Deterministic injuries (100%) yield lower variance than stochastic (50%)

## 6.2 Key Takeaways

From a learning perspective, this project taught me:

- **MARL is hard**: Getting MAPPO to learn anything meaningful took far longer than anticipated. The sensitivity to hyperparameters and environment configuration was surprising.

- The critical importance of matching parallel environments, episode lengths, and batch sizes—a lesson learned through many failed experiments.

- Preliminary evidence that richer observations may not always help decentralized multi-agent learning, though this finding needs verification with complete data.

- The tendency of policies to converge to defensive local optima when scoring is difficult, highlighting the challenge of sparse rewards even with dense shaping.

## 6.3 Future Work

Several promising directions remain unexplored:

1. **Curriculum Learning**: Gradually increasing opponent strength and injury probability during training may help agents learn offensive strategies before settling into defensive optima.

2. **Recurrent Architectures**: LSTMs or GRUs could better handle the partial observability inherent in injury scenarios, especially for the "none" observation mode.

3. **Alternative Algorithms**: Comparing MAPPO with HAPPO, QMIX, or IPPO under injury conditions would reveal algorithm-specific robustness properties.

4. **Communication**: Allowing agents to explicitly communicate injury status (rather than passively observing it) may enable more sophisticated coordination.

5. **Self-Play**: Training against learned opponents rather than heuristic AI would create a more challenging and dynamic training curriculum.

6. **Injury Severity Curves**: Varying the speed penalty from mild (80% speed) to severe (20% speed) could reveal thresholds where team performance collapses.

### 6.4 Reproducibility

All code I used is available at:
https://github.com/HasanFakih21/COMP4900-Project

The implementation builds on BenchMARL [3] and VMAS [2]. Experiments were conducted on a single GPU (CUDA) with each run taking approximately 1 hour for 6 million environment frames. Note that due to the iterative nature of hyperparameter tuning, exact reproduction of the baseline stabilization process may be difficult.

## References


[1] Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A., & Wu, Y. (2022). The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. *Advances in Neural Information Processing Systems*, 35.

[2] Bettini, M., Kortvelesy, R., Blumenkamp, J., & Prorok, A. (2022). VMAS: A Vectorized Multi-Agent Simulator for Collective Robot Learning. *Proceedings of the 16th International Symposium on Distributed Autonomous Robotic Systems (DARS).*

[3] Bettini, M., Prorok, A., & Moens, V. (2024). BenchMARL: Benchmarking Multi-Agent Reinforcement Learning. *Journal of Machine Learning Research*, 25(217), 1–10.

[4] Canaan, R., Salge, C., Togelius, J., & Nealen, A. (2019). Learning Complex Teamwork Tasks Using a Given Sub-task Decomposition. *arXiv preprint arXiv:1906.08205.*

# A Unstable Training Example

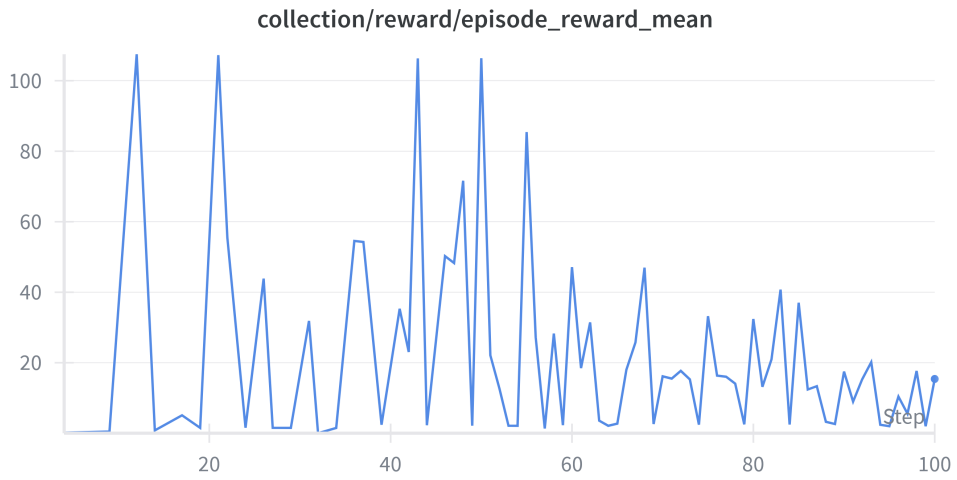


Figure 4: Training instability in early experiments with misaligned batch/episode configuration. Episode mean returns oscillate dramatically without corresponding behaviour improvements, indicating failed learning.